

**TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHẦN HIỆU TẠI TP.HCM
BỘ MÔN CÔNG NGHỆ THÔNG TIN**



JUnit 5 cơ bản

Buổi 5 / 12 — Môn: Kiểm thử Phần mềm

Học kỳ 2025–2026

Nội dung buổi học

- 1 JUnit là gì? Lịch sử và kiến trúc JUnit 5
- 2 Các bước cài đặt JUnit: Maven và Gradle (DigiShield)
- 3 Cấu trúc thư mục test và test đầu tiên
- 4 Vòng đời test và các annotation cơ bản
- 5 Họ assertion của JUnit 5
- 6 AssertJ — assertion kiểu fluent
- 7 Ca kiểm thử hoàn chỉnh với StubApiClient
- 8 Parameterized tests
- 9 Chạy test bằng Gradle và đọc report
- 10 Thiết kế dễ kiểm thử (testability) — Thực hành — Bài nộp

Vị trí trong đề cương môn học

Chương 4 — JUnit trong kiểm thử hộp trắng

Nội dung chương theo đề cương: **Giới thiệu** — **Khái niệm chung** — **Các bước cài đặt JUnit**.

Buổi	Chương	Chủ đề
3–4	C3 Hộp trắng	CFG, độ phủ, cyclomatic, JaCoCo
5	C4 JUnit	JUnit 5 cơ bản (hôm nay)
6	C4 JUnit	Mockito: mock / stub / verify
7	C4 (mở rộng)	Integration test: Spring, Testcontainers
8–9	C5 Hộp đen	EP, BVA, bảng quyết định, Postman

Kết nối với buổi 3–4

Buổi 3–4 ta đã **thiết kế** test case trên giấy từ CFG và độ phủ nhánh. Hôm nay ta **cài đặt** chúng thành mã chạy được bằng JUnit 5 trên chính codebase DigiShield.

Mục tiêu buổi học

Sau buổi học, sinh viên có thể:

- Trình bày được kiến trúc JUnit 5 (Platform / Jupiter / Vintage)
- Cài đặt JUnit 5 vào project Maven và project Gradle
- Giải thích cấu hình test thật của DigiShield (spring-boot-starter-test, JVM Test Suites)
- Viết unit test theo mẫu AAA với @Test, các annotation vòng đời, assertion và AssertJ
- Viết parameterized test với @ValueSource, @CsvSource, @MethodSource
- Chạy test bằng ./gradlew test và đọc report HTML

Chuẩn bị

Clone repo DigiShield, mở bằng IntelliJ IDEA; JDK 25; không cần Docker cho buổi này.

Vì sao phải tự động hóa unit test?

Kiểm thử thủ công

- Chạy app, nhập tay, quan sát mắt thường
- Lặp lại sau **mỗi lần sửa code** — tốn công
- Không đo được độ phủ
- Dễ bỏ sót trường hợp biên

Unit test tự động

- Viết một lần, chạy hàng nghìn lần
- Chạy trong CI/CD trước khi merge
- Là “lưới an toàn” khi refactor
- Là **tài liệu sống** (living documentation) mô tả hành vi code

Trong DigiShield

Backend hiện có **176 test** trong 27 class test. Mỗi lần đẩy code, toàn bộ chạy lại bằng `./gradlew test` — vài chục giây thay vì vài giờ kiểm thử tay.

JUnit là gì?

Định nghĩa

JUnit là **framework kiểm thử đơn vị** (unit testing framework) cho Java: cung cấp annotation để đánh dấu test, assertion để kiểm tra kết quả, và bộ chạy (runner) để thực thi + báo cáo.^[5]

Lịch sử:

- 1997: Kent Beck & Erich Gamma viết JUnit trên chuyến bay tới hội nghị OOPSLA
- Khởi nguồn họ framework **xUnit** (NUnit, PyTest, PHPUnit, ...)
- 2006: JUnit 4 (annotation `@Test`)
- 2017: JUnit 5 — viết lại toàn bộ

Vị thế hiện nay:

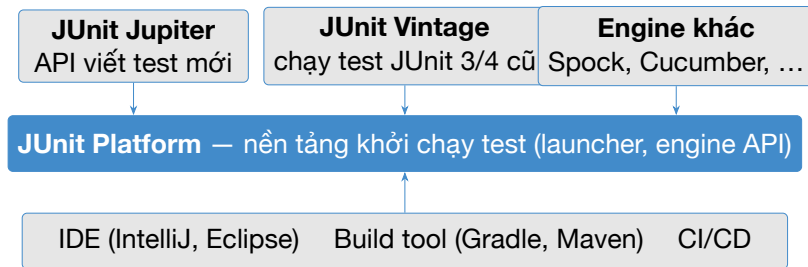
- Chuẩn *de facto* cho test Java
- Tích hợp sẵn trong Maven, Gradle, IntelliJ, Eclipse, CI/CD
- `spring-boot-starter-test` chọn JUnit 5 làm mặc định

JUnit 4 → JUnit 5: có gì thay đổi?^[5]

	JUnit 4 (2006)	JUnit 5 (2017–nay)
Kiến trúc	1 file jar duy nhất	3 phần: Platform + Jupiter + Vintage
Java tối thiểu	Java 5	Java 8+ (lambda)
Trước/sau mỗi test	@Before/@After	@BeforeEach/@AfterEach
Trước/sau tất cả	@BeforeClass/@AfterClass	@BeforeAll/@AfterAll
Bỏ qua test	@Ignore	@Disabled
Mở rộng	@RunWith (1 runner)	@ExtendWith (nhiều extension)
Test theo tham số	Cổng kênh (runner riêng)	@ParameterizedTest tích hợp
Test lồng nhau	Không	@Nested

Lưu ý khi đọc tài liệu trên mạng: nhiều bài viết cũ vẫn dùng @Before, @RunWith của JUnit 4 — các annotation này **không tồn tại** trong JUnit 5 (Jupiter).

Kiến trúc JUnit 5 = Platform + Jupiter + Vintage



- **Platform:** giao tiếp với IDE/Gradle; khám phá và khởi chạy test
- **Jupiter:** annotation (`@Test`, `@BeforeEach`, ...) và assertion ta dùng hằng ngày
- **Vintage:** cầu nối chạy test viết bằng JUnit 3/4 trên Platform mới

Các bước cài đặt JUnit — tổng quan

- ❶ **Khai báo dependency** JUnit Jupiter trong file build (`pom.xml` với Maven, `build.gradle.kts` với Gradle)
- ❷ **Bảo đảm build tool dùng JUnit Platform** để chạy test (Maven Surefire 2.22+; Gradle `useJUnitPlatform()` hoặc `useJUnitJupiter()`)
- ❸ **Tạo thư mục** `src/test/java` song song với `src/main/java`
- ❹ **Viết class test** theo quy ước `*Test.java`
- ❺ **Chạy**: `mvn test` / `./gradlew test` / nút Run trong IDE

Hai con đường trong môn học

Maven: cho project riêng của nhóm (nếu nhóm tự chọn đề tài).

Gradle: theo đúng cấu hình thật của DigiShield — dùng cho BTL.

Cài đặt với Maven — pom.xml

```
1 <!-- 1. Dependency JUnit Jupiter (scope test) -->
2 <dependency>
3   <groupId>org.junit.jupiter</groupId>
4   <artifactId>junit-jupiter</artifactId>
5   <version>5.11.4</version>
6   <scope>test</scope>
7 </dependency>
8
9 <!-- 2. Surefire 2.22+ de hỗ trợ JUnit Platform -->
10 <plugin>
11   <groupId>org.apache.maven.plugins</groupId>
12   <artifactId>maven-surefire-plugin</artifactId>
13   <version>3.5.2</version>
14 </plugin>
```

Chạy: `mvn test`. Với project Spring Boot chỉ cần `spring-boot-starter-test` — đã gói sẵn JUnit 5.

Cài đặt với Gradle — project thường

```
// build.gradle.kts
dependencies {
    // BOM giu cac artifact JUnit cung phien ban
    testImplementation(platform(
        "org.junit:junit-bom:5.11.4"))
    testImplementation(
        "org.junit.jupiter:junit-jupiter")
    testRuntimeOnly(
        "org.junit.platform:junit-platform-launcher")
}

tasks.test {
    // Bat buoc: chay test tren JUnit Platform
    useJUnitPlatform()
}
```

Thiếu `useJUnitPlatform()` là lỗi kinh điển: Gradle biên dịch test nhưng **không chạy** test nào cả.

Cài đặt trong DigiShield — convention cho 9 module

Trích buildSrc/src/main/kotlin/digishield.spring-module-conventions.gradle.kts

```
dependencies {  
    // ... dependency production ...  
    "testImplementation"("org.springframework.boot:"  
        + "spring-boot-starter-test")  
    "testImplementation"("org.springframework.modulith:"  
        + "spring-modulith-starter-test")  
    "testRuntimeOnly"("org.junit.platform:"  
        + "junit-platform-launcher")  
}  
  
tasks.withType<Test> {  
    useJUnitPlatform()  
}
```

Convention plugin áp dụng cho cả 9 module nghiệp vụ (auth, ai, reporting, ...) — khai báo **một lần**, mọi module đều có JUnit 5.

spring-boot-starter-test kéo theo những gì?

Thư viện	Vai trò
JUnit 5 (Jupiter)	Framework test: <code>@Test</code> , assertion, lifecycle
Mockito	Tạo mock/stub cho dependency (buổi 6)
AssertJ	Assertion kiểu fluent: <code>assertThat(x).isEqualTo(y)</code>
Hamcrest	Matcher (ít dùng khi đã có AssertJ)
JSONassert + JsonPath	So sánh và trích xuất JSON
Spring Test	<code>@SpringBootTest</code> , <code>MockMvc</code> (buổi 7)

Ý nghĩa

Chỉ cần **một** dependency, sinh viên có đủ toàn bộ công cụ của cả 3 buổi 5–6–7. Không cần khai báo JUnit/Mockito/AssertJ riêng lẻ.

DigiShield: JVM Test Suites — tách unit và integration^[7]

Trích boot/app/build.gradle.kts (cấu hình thật)

```
testing {
    suites {
        // Suite unit test mac dinh -> JUnit Jupiter
        getByName<JvmTestSuite>("test") {
            useJUnitJupiter()
        }
        // Suite integration test: Spring + Testcontainers
        register<JvmTestSuite>("integrationTest") {
            useJUnitJupiter()
            // ... dependency rieng: Testcontainers, ...
            targets { all { testTask.configure {
                // IT cham (can Docker) chay sau UT nhanh
                shouldRunAfter(tasks.named("test"))
            } } }
        }
    }
}
```

Hai tầng test của DigiShield

	Unit test (buổi 5–6)	Integration test (buổi 7)
Lệnh chạy	<code>./gradlew test</code>	<code>./gradlew integrationTest</code>
Thư mục	<code>src/test</code>	<code>boot/app/src/integrationTest</code>
Quy ước tên	<code>*Test.java</code>	<code>*IT.java</code>
Phạm vi	1 class, cô lập	Nhiều module + DB thật
Hạ tầng	Không cần gì	Docker (Testcontainers, PostgreSQL 16)
Tốc độ	mili-giây / test	giây-phút / test
Ví dụ thật	<code>StubAiClientTest</code>	<code>TenantIsolationIT</code>

Liên hệ testing pyramid (buổi 2): đáy tháp là unit test — nhiều, nhanh, rẻ. DigiShield tách hẳn hai suite để CI chạy tầng nhanh trước, tầng chậm sau.

Cấu trúc thư mục: code và test song song

```
modules/ai/  
  src/main/java/  
    com/digishield/ai/application/  
      StubAiClient.java          <-- ma nguon  
  src/test/java/  
    com/digishield/ai/application/  
      StubAiClientTest.java      <-- unit test
```

Quy ước:

- Test đặt **cùng package** với class được test
- Tên file: <TenClass>Test.java
- Mỗi class quan trọng một class test

Lợi ích cùng package:

- Truy cập được method *package-private*
- Không cần import class được test
- Dễ tìm: IDE nhảy qua lại bằng Ctrl+Shift+T

Hàm sẽ kiểm thử: ageLabel (module reporting)

Trích ReportingServiceImpl.java (đã bỏ private để minh họa) — nhãn “tuổi” của báo cáo lừa đảo

```
1 String ageLabel(Instant reportedAt, Instant now) {  
2     if (reportedAt == null) {  
3         return null;  
4     }  
5     Duration d = Duration.between(reportedAt, now);  
6     long mins = Math.max(0, d.toMinutes());  
7     if (mins < 60) {  
8         return mins + "p";    // vd "5p"   (p = phut)  
9     }  
10    long hours = d.toHours();  
11    if (hours < 24) {  
12        return hours + "h";    // vd "3h"  
13    }  
14    return d.toDays() + "d";    // vd "2d"  
15 }
```

Hàm **thuần** (pure): kết quả chỉ phụ thuộc tham số — ứng viên hoàn hảo cho unit test đầu tiên.

Test đầu tiên: @Test + assertEquals

```
1 package com.digishield.reporting.application;
2
3 import java.time.Duration;
4 import java.time.Instant;
5
6 import org.junit.jupiter.api.Test;
7 import static org.junit.jupiter.api.Assertions.*;
8
9 class ReportAgeTest {
10
11     @Test
12     void ageLabel_under60Minutes_returnsMinutes() {
13         Instant now =
14             Instant.parse("2026-07-14T10:00:00Z");
15         Instant reportedAt =
16             now.minus(Duration.ofMinutes(5));
17
18         String label = ReportAge.ageLabel(reportedAt, now);
19
20         assertEquals("5p", label);
21     }
22 }
```

@Test đánh dấu method là một ca kiểm thử; assertEquals(expected, actual) – chú ý thứ tự tham số! ReportAge là class ta sẽ tách ra từ ReportingServiceImpl (xem slide sau).

Mẫu AAA: Arrange — Act — Assert

Ba bước của mọi unit test

- **Arrange:** chuẩn bị dữ liệu, đối tượng, trạng thái
- **Act:** gọi **một** hành vi cần kiểm thử
- **Assert:** kiểm tra kết quả so với kỳ vọng

Mẹo

Tách 3 khối bằng dòng trống. Nếu phần Act có *nhiều hơn một* lời gọi hành vi — cân nhắc tách test.

```
@Test
void ageLabel_over24Hours_returnsDays() {
    // ARRANGE: bao cao gui cach day 2 ngay
    Instant now = Instant.parse(
        "2026-07-14T10:00:00Z");
    Instant reportedAt =
        now.minus(Duration.ofDays(2));

    // ACT: goi ham can kiem thu
    String label =
        ReportAge.ageLabel(reportedAt, now);

    // ASSERT: ky vong nhan "2d"
    assertEquals("2d", label);
}
```

Quy ước đặt tên method test

Mẫu khuyến nghị trong môn học

`tenMethod_dieuKien_ketQuaKyVong`

Tên test (thật trong DigiShield)	Đọc thành câu
<code>classify_flagsThreatSignals</code>	classify gắn nhãn threat khi có tín hiệu lừa đảo
<code>moderate_passesSafeContent</code>	moderate cho qua nội dung an toàn
<code>classify_returnsCleanForBenignContent</code>	classify trả nhãn clean với nội dung lành tính
<code>benchmark_whenNoScores_returnsZero</code>	benchmark trả 0 khi chưa có điểm nào

Nguyên tắc: tên test là **đặc tả hành vi** — người đọc hiểu code làm gì mà không cần mở phần thân. Tránh tên vô nghĩa: `test1`, `test0k`.

Ghi chú: test method private thế nào?

Vấn đề thực tế

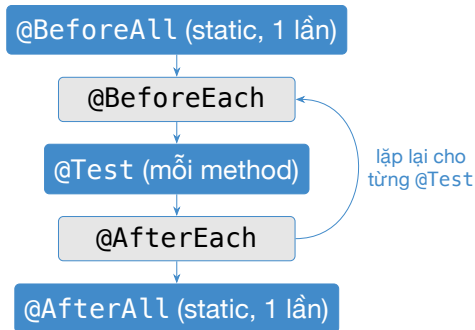
Trong code thật, `ageLabel` là method **private** của `ReportingServiceImpl`. JUnit không gọi trực tiếp được method private.

Các lựa chọn (từ tốt đến kém):

- 1 **Test gián tiếp** qua method public đang gọi nó (`listPhishingReports` trả DTO chứa nhãn tuổi)
- 2 **Tách** logic thuần ra class riêng (`ReportAge`) — vừa dễ test, vừa tái sử dụng
- 3 Hạ xuống **package-private** (bỏ từ khóa `private`) — test cùng package gọi được; chấp nhận được với hàm thuần
- 4 Reflection — *không khuyến khích*: test vỡ khi đổi tên

Bài học thiết kế: khó viết test thường là **triệu chứng** của thiết kế khó bảo trì. Ta quay lại chủ đề này ở phần *testability* cuối buổi.

Vòng đời một class test JUnit 5



- Mặc định JUnit tạo **instance mới** của class test cho **mỗi** test method $\Rightarrow$ test độc lập, không rò trạng thái
- `@BeforeAll`/`@AfterAll` phải là `static`: chạy trước khi có instance nào

Vòng đời trong test thật của DigiShield

Trích AnalyticsServiceImplTest.java (module analytics)

```
1 class AnalyticsServiceImplTest {
2
3     private static final UUID TENANT_ID = UUID.fromString(
4         "11111111-1111-1111-1111-111111111111");
5
6     @BeforeEach
7     void setUp() {
8         // Mọi test cần tenant trong ThreadLocal
9         TenantContext.set(TENANT_ID.toString());
10    }
11
12    @AfterEach
13    void tearDown() {
14        // Don sach de test sau không bị ảnh hưởng
15        TenantContext.clear();
16    }
17 }
```

Quên tearDown ⇒ tenant “rò” sang test khác — lỗi chập chờn (flaky) rất khó truy.

@DisplayName, @Disabled, @Tag

```
@Test
@DisplayName(
    "classify: co tu 'password' -> threat")
void classify_flagsThreatSignals() { }

@Test
@Disabled("Cho fix bug DS-142")
void moderate_handlesEmoji() { }

@Test
@Tag("slow")
void classify_largePayload() { }
```

- @DisplayName: tên đẹp hiển thị trong report/IDE
- @Disabled: tắt tạm — **luôn kèm lý do**; test bị tắt vẫn hiện trong report (skipped)
- @Tag: gắn nhãn để lọc khi chạy (vd chỉ chạy nhóm "slow" trên CI ban đêm)

Cảnh báo

@Disabled không phải thùng rác: test tắt quá lâu = mất một phần lưới an toàn.

Họ assertion của JUnit 5 — tổng quan

Assertion	Kiểm tra	Ví dụ dùng khi
<code>assertEquals(e, a)</code>	Giá trị bằng nhau	Nhãn phân loại, số điểm
<code>assertTrue / assertFalse</code>	Điều kiện Boolean	confidence ≥ 0.6
<code>assertNull / assertNotNull</code>	null hay không	DTO trả về tồn tại
<code>assertThrows</code>	Ném đúng exception	Email blank bị từ chối
<code>assertAll</code>	Nhóm assertion, chạy hết	Kiểm nhiều field một DTO
<code>assertTimeout</code>	Hoàn thành trong hạn	Hàm không được quá chậm

Import tĩnh: `import static org.junit.jupiter.api.Assertions.*;`

Mọi assertion đều nhận thêm tham số cuối là **thông điệp** hiển thị khi fail:

`assertEquals("5p", label, "nhan phut sai").`

assertEquals / assertTrue / assertNotNull

```
1 StubAiClient client = new StubAiClient();
2
3 @Test
4 void classify_twoThreatSignals_confidenceGrows() {
5     // "otp" va "password" -> 2 tin hieu lua dao
6     ClassificationView r = client.classify(
7         "Nhap OTP va password de mo khoa tai khoan");
8
9     assertEquals("threat", r.label());
10    assertTrue(r.confidence() >= 0.6,
11        "confidence phai tu 0.6 tro len");
12    assertNotNull(r.reason());
13 }
```

Bắt số thực

Không dùng `assertEquals(0.8, r.confidence())` với double — sai số dấu phẩy động! Dùng `assertEquals(0.8, x, 1e-9)` (tham số delta) hoặc `AssertJ isBetween`.

assertThrows — kiểm thử tình huống lỗi

Hành vi thật: AuthServiceImpl.createUser từ chối email trống

```
1 @Test
2 void createUser_blankEmail_throwsException() {
3     var request = requestWithEmail(" ");
4
5     IllegalArgumentException ex = assertThrows(
6         IllegalArgumentException.class,
7         () -> authService.createUser(request));
8
9     assertEquals("email is required",
10        ex.getMessage());
11 }
```

- Tham số 2 là **lambda**: code ném exception được bọc lại, JUnit bắt và so khớp kiểu
- **assertThrows trả về** exception $\Rightarrow$ kiểm tiếp message
- Test fail nếu: không ném gì, hoặc ném sai kiểu

assertAll — kiểm nhiều field, bảo đảm mọi lỗi

```
1 @Test
2 void classify_threat_allFieldsValid() {
3     ClassificationView r = client.classify(
4         "urgent: verify your account now");
5
6     assertAll("classification",
7         () -> assertEquals("threat", r.label()),
8         () -> assertTrue(r.confidence() >= 0.6),
9         () -> assertTrue(r.confidence() <= 1.0),
10        () -> assertNotNull(r.reason()));
11 }
```

Không có assertAll

Assertion đầu fail $\Rightarrow$ dừng ngay, các lỗi sau bị **che khuất**; phải sửa-chạy nhiều vòng.

Có assertAll

Chạy **tất cả** lambda, gom mọi lỗi vào một `MultipleFailuresError` — thấy toàn cảnh trong 1 lần chạy.

assertTimeout — ràng buộc thời gian

```
1 @Test
2 void classify_completesWithin200ms() {
3     ClassificationView r = assertTimeout(
4         Duration.ofMillis(200),
5         () -> client.classify("hello team"));
6
7     assertEquals("clean", r.label());
8 }
```

- Lambda được chạy; nếu vượt hạn $\Rightarrow$ test fail (vẫn chờ chạy xong)
- `assertTimeoutPreemptively`: ngắt ngay khi quá hạn (chạy ở thread khác — cẩn thận với `ThreadLocal` như `TenantContext`!)

Dùng có chừng mực

Timeout phụ thuộc máy chạy $\Rightarrow$ dễ flaky trên CI yếu. Kiểm thử hiệu năng nghiêm túc để dành đến buổi 12 (JMeter, k6).

AssertJ — assertion kiểu fluent

AssertJ là gì?^[6]

Thư viện assertion đi kèm spring-boot-starter-test, cú pháp **chuỗi** (fluent): `assertThat(actual).isEqualTo(expected)`. DigiShield dùng AssertJ trong **toàn bộ** test hiện có.

Vì sao đội DigiShield chọn AssertJ?

- Đọc như câu văn: `assertThat(reasons).hasSize(2)`
- Không nhầm thứ tự expected/actual như `assertEquals`
- IDE gợi ý đúng method theo **kiểu dữ liệu** (String có `isNotBlank`, List có `contains`, ...)
- Thông báo lỗi giàu ngữ cảnh: in phần tử thiếu/thừa của collection

Import duy nhất cần nhớ

```
import static org.assertj.core.api.Assertions.assertThat;
```

JUnit Assertions vs AssertJ — cùng một ý

JUnit (Assertions)

```
assertEquals("threat", r.label());

assertTrue(r.confidence() >= 0.0
    && r.confidence() <= 1.0);

assertTrue(reasons.size() >= 2);

assertTrue(
    reasons.get(0).contains("malware"));

assertFalse(r.reason().isBlank());
```

AssertJ (fluent)

```
assertThat(r.label())
    .isEqualTo("threat");

assertThat(r.confidence())
    .isBetween(0.0, 1.0);

assertThat(reasons)
    .hasSizeGreaterThanOrEqualTo(2);

assertThat(reasons.get(0))
    .contains("malware");

assertThat(r.reason()).isNotBlank();
```

Khi fail, AssertJ in: *Expecting actual ... to contain "malware" but did not* — kèm giá trị thực tế, đỡ phải debug.

AssertJ — các method hay dùng nhất (1/2)

Kiểu	Method	Ví dụ
Mọi kiểu	<code>isEqualTo</code> , <code>isNotNull</code>	<code>assertThat(label)</code> <code>.isEqualTo("spam")</code>
String	<code>contains</code> , <code>startsWith</code> , <code>isNotBlank</code>	<code>assertThat(ref)</code> <code>.startsWith("tmpl/")</code>
Số	<code>isBetween</code> , <code>isZero</code> , <code>isPositive</code>	<code>assertThat(c)</code> <code>.isBetween(0.0, 1.0)</code>
Collection	<code>hasSize</code> , <code>isEmpty</code> , <code>contains</code>	<code>assertThat(reasons)</code> <code>.isEmpty()</code>
Exception	<code>assertThatThrownBy</code>	<code>...assertInstanceOf(</code> <code>IllegalArgumentException</code> <code>.class)</code>

AssertJ — các method hay dùng nhất (2/2)

Quy ước trong môn học (theo codebase DigiShield)

Viết test mới bằng **AssertJ**. Vẫn dùng JUnit `assertThrows/assertAll` khi tiện — hai thư viện chung sống thoải mái trong cùng một test.

Test thật trong DigiShield: StubAiClientTest

Trích nguyên văn modules/ai/src/test/java/.../StubAiClientTest.java

```
1 class StubAiClientTest {
2
3     private final StubAiClient client =
4         new StubAiClient();
5
6     @Test
7     void classify_flagsThreatSignals() {
8         ClassificationView result = client.classify(
9             "Please verify your account and "
10            + "enter your password");
11
12         assertThat(result.label())
13             .isEqualTo("threat");
14         assertThat(result.confidence())
15             .isBetween(0.0, 1.0);
16         assertThat(result.reason()).isNotBlank();
17     }
18 }
```

Đối tượng kiểm thử: StubAiClient.classify

modules/ai/.../StubAiClient.java — bộ phân loại nội dung lừa đảo (rút gọn)

```
1 public ClassificationView classify(String payload) {
2     String text = payload == null
3         ? "" : payload.toLowerCase(Locale.ROOT);
4
5     long threatHits = countHits(text, THREAT_SIGNALS);
6     long spamHits   = countHits(text, SPAM_SIGNALS);
7
8     if (threatHits > 0) {
9         double confidence =
10             clamp(0.6 + 0.1 * threatHits);
11         return new ClassificationView(
12             "threat", confidence, "...");
13     }
14     if (spamHits > 0) {
15         return new ClassificationView("spam",
16             clamp(0.5 + 0.1 * spamHits), "...");
17     }
18     return new ClassificationView("clean", 0.92, "...");
19 }
```

THREAT_SIGNALS: "password", "verify your account", "otp", "urgent", ...;

SPAM_SIGNALS: "sale", "free", "%", ...

Đối tượng kiểm thử: StubAiClient.moderate

```
1 public ModerationView moderate(String content) {  
2     String text = content == null  
3         ? "" : content.toLowerCase(Locale.ROOT);  
4  
5     List<String> reasons = new ArrayList<>();  
6     for (String word : UNSAFE_WORDS) {  
7         if (text.contains(word)) {  
8             reasons.add("Nội dung chưa tu: " + word);  
9         }  
10    }  
11    String verdict;  
12    if (reasons.isEmpty())      verdict = "pass";  
13    else if (reasons.size() >= 2) verdict = "block";  
14    else                       verdict = "flag";  
15    return new ModerationView(  
16        verdict, List.copyOf(reasons));  
17 }
```

Ba lớp hành vi rõ rệt: 0 lý do $\rightarrow$ pass; 1 $\rightarrow$ flag; $\geq 2 \rightarrow$ block — tương ứng ba nhánh code, đúng kiểu độ phủ nhánh đã học ở hộp trắng.

Bộ test hoàn chỉnh (1/2): hai nhánh của `classify`

```
1 class StubAiClientTest {
2     private final StubAiClient client =
3         new StubAiClient();
4
5     @Test
6     void classify_flagsThreatSignals() {
7         ClassificationView r = client.classify(
8             "Tai khoan suspended, click here ngay");
9         assertThat(r.label()).isEqualTo("threat");
10        assertThat(r.confidence())
11            .isBetween(0.6, 1.0); // 2 hit -> ~0.8
12    }
13
14    @Test
15    void classify_returnsCleanForBenignContent() {
16        assertThat(client.classify(
17            "Hen gap ban o hop giao ban sang mai")
18            .label()).isEqualTo("clean");
19    }
20 }
```

Bộ test hoàn chỉnh (2/2): ba lớp của moderate

```
1  @Test
2  void moderate_passesSafeContent() {
3      ModerationView r = client.moderate("hello world");
4      assertThat(r.verdict()).isEqualTo("pass");
5      assertThat(r.reasons()).isEmpty();
6  }
7
8  @Test
9  void moderate_flagsSingleUnsafeWord() {
10     ModerationView r = client.moderate(
11         "canh bao: phat hien malware");
12     assertThat(r.verdict()).isEqualTo("flag");
13     assertThat(r.reasons()).hasSize(1);
14 }
15
16 @Test
17 void moderate_blocksMultipleUnsafeWords() {
18     ModerationView r = client.moderate(
19         "ransomware nay co the exploit he thong");
20     assertThat(r.verdict()).isEqualTo("block");
21     assertThat(r.reasons())
22         .hasSizeGreaterThanOrEqualTo(2);
23 }
24 }
```

Phân tích: mỗi test bao phủ một hành vi

Test	Dữ liệu vào	Nhánh code	Kỳ vọng
<code>classify_flagsThreat...</code>	"suspended", "click here"	<code>threatHits > 0</code>	threat
<code>classify_returnsClean...</code>	câu vô hại	cả hai đếm = 0	clean
<code>moderate_passes...</code>	"hello world"	reasons rỗng	pass
<code>moderate_flags...</code>	1 từ ("malware")	<code>size == 1</code>	flag
<code>moderate_blocks...</code>	2 từ	<code>size >= 2</code>	block

Liên hệ buổi 3–4

Bảng trên chính là **ánh xạ test** ↔ **nhánh**: 5 test này đủ đạt độ phủ nhánh cao cho hai method. Còn thiếu nhánh nào? (gợi ý: `payload == null`, nhánh *spam*) — đó là bài thực hành.

Vấn đề: nhiều test chỉ khác dữ liệu

```
@Test
void ageLabel_0min() {
    ... assertEquals("0p", ...); }
@Test
void ageLabel_59min() {
    ... assertEquals("59p", ...); }
@Test
void ageLabel_60min() {
    ... assertEquals("1h", ...); }
// ... lap lai 6 lan, chi khac so lieu
```

Code smell trong test

Copy-paste test chỉ đổi số liệu:

- Dài dòng, khó đọc
- Sửa logic phải sửa n chỗ
- Dễ quên bổ sung ca biên mới

Giải pháp

@ParameterizedTest: **một** method, **nhiều** bộ dữ liệu — mỗi bộ hiện thành một test riêng trong report.

@ValueSource — một tham số đơn

```
1 @ParameterizedTest
2 @ValueSource(strings = {
3     "malware", "ransomware", "ddos", "exploit"})
4 void moderate_singleUnsafeWord_flags(String word) {
5     ModerationView r = client.moderate(
6         "canh bao: phat hien " + word);
7
8     assertThat(r.verdict()).isEqualTo("flag");
9     assertThat(r.reasons()).hasSize(1);
10 }
```

- 4 giá trị ⇒ chạy thành **4 test** độc lập; một giá trị fail không chặn các giá trị khác
- @ValueSource hỗ trợ strings, ints, longs, doubles, booleans, ...
- Nhớ đổi @Test thành @ParameterizedTest

@CsvSource — bảng dữ liệu vào/ra cho ageLabel

```
1 @ParameterizedTest(name = "{0} phut truooc -> {1}")
2 @CsvSource({
3     "0,      0p",    // bien duoi
4     "59,     59p",   // sat bien 60 phut
5     "60,     1h",    // vua cham nguong gio
6     "1439,   23h",   // sat bien 24 gio
7     "1440,   1d",    // vua cham nguong ngay
8     "2880,   2d"
9 })
10 void ageLabel_byMinutes(long minutes, String expected) {
11     Instant now =
12         Instant.parse("2026-07-14T10:00:00Z");
13     Instant reportedAt =
14         now.minus(Duration.ofMinutes(minutes));
15
16     assertEquals(expected,
17         ReportAge.ageLabel(reportedAt, now));
18 }
```

Mỗi dòng CSV là một ca; giá trị tự chuyển kiểu theo tham số. Nhận ra gì không? — đây chính là **BVA** quanh biên 60 và 1440!

@MethodSource — dữ liệu phức tạp từ method

```
1 static Stream<Arguments> classifyCases() {
2     return Stream.of(
3         Arguments.of(
4             "nhap password de xac minh", "threat"),
5         Arguments.of(
6             "giam gia 50% cuoi tuan nay", "spam"),
7         Arguments.of(
8             "hop giao ban 9h sang mai", "clean"));
9 }
10
11 @ParameterizedTest(name = "\"{0}\" -> {1}")
12 @MethodSource("classifyCases")
13 void classify_labelsByContent(
14     String payload, String expected) {
15     assertThat(client.classify(payload).label())
16         .isEqualTo(expected);
17 }
```

Method nguồn: static, trả Stream<Arguments> — dùng khi dữ liệu là object, không nhét vừa chuỗi CSV.

Khi nào dùng nguồn nào?

Nguồn	Phù hợp khi	Ví dụ trong bài
@ValueSource	1 tham số kiểu đơn giản	4 từ khóa không an toàn
@CsvSource	Bảng vào/ra 2–4 cột gọn	phút → nhãn tuổi
@MethodSource	Object phức tạp, dữ liệu tính toán	payload → nhãn phân loại
@EnumSource	Duyệt mọi giá trị enum	mọi TemplateChannel
@CsvFileSource	Bảng lớn để trong file .csv	bộ test case của BTL

Đừng lạm dụng

Parameterized test phù hợp khi các ca **cùng logic kiểm tra**. Nếu mỗi ca cần assert khác nhau (pass/flag/block với field khác nhau) — viết test thường rõ ràng hơn.

Chạy test bằng Gradle

```
# Toàn bộ unit test của cả project (176 test)
./gradlew test

# Chỉ test của module ai
./gradlew :modules:ai:test

# Chỉ một class test
./gradlew :modules:ai:test \
    --tests "*StubAiClientTest"

# Chỉ một method test
./gradlew :modules:ai:test --tests \
    "*StubAiClientTest.classify_flagsThreatSignals"

# Integration test (bước 7 - cần Docker)
./gradlew :boot:app:integrationTest
```

Gradle chỉ chạy lại test khi code đổi (up-to-date check); ép chạy lại: thêm `--rerun-tasks` hoặc `cleanTest`.

Đọc report HTML của Gradle

Vị trí report sau khi chạy `./gradlew :modules:ai:test`

`modules/ai/build/reports/tests/test/index.html`

Report cho biết:

- Tổng số test / fail / skipped, thời gian chạy
- Duyệt theo package → class → method
- Với test fail: stack trace + thông điệp assertion
- Parameterized test: từng bộ dữ liệu một dòng

Khi test fail trên terminal:

- Gradle in tên test fail + 1 dòng lý do
- Mở link report ở cuối log để xem chi tiết
- Trong IDE: click chạy từng test, xanh/đỏ ngay lập tức

Nhắc lại buổi 4

Chạy test xong, JaCoCo tự sinh report độ phủ: `build/reports/jacoco/`;
report gộp toàn project: `./gradlew testCodeCoverageReport`.

Chạy test trong IntelliJ IDEA

Thao tác cơ bản:

- Icon ▶ xanh bên trái class/method → Run
- `Ctrl+Shift+F10`: chạy test dưới con trỏ
- `Shift+F10`: chạy lại cấu hình gần nhất
- Chuột phải thư mục `src/test/java` → Run All Tests

Nên bật:

- Run with Coverage — xem độ phủ ngay trong editor
- Rerun Failed Tests — chỉ chạy lại test đỏ
- Debug test: đặt breakpoint trong test *lần* trong code được test

Quy trình khuyến nghị trên lớp

Viết test → chạy bằng IDE cho nhanh → trước khi commit chạy `./gradlew test` để chắc chắn giống môi trường CI.

Vì sao ageLabel nhận now làm tham số?

```
// KHO test: tu lay gio he thong
String ageLabel(Instant reportedAt) {
    Duration d = Duration.between(
        reportedAt, Instant.now());
    // ket qua doi theo... gio chay test!
}

// DE test: 'now' la tham so
// (cach DigiShield dang lam)
String ageLabel(Instant reportedAt,
                Instant now) {
    Duration d = Duration.between(
        reportedAt, now);
}
```

Nếu gọi `Instant.now()` bên trong

- Test "5 phút trước" hôm nay pass, mai...vẫn pass, nhưng test biên "59/60 phút" thành **flaky**
- Không thể test ca "2 ngày trước" mà không chờ 2 ngày

Nguyên tắc

Phụ thuộc ẩn (giờ hệ thống, random, biến toàn cục) → đưa thành **tham số** hoặc dependency được inject.

Tổng quát hóa: inject java.time.Clock

```
1 class ReportingServiceImpl {
2     private final Clock clock; // inject qua constructor
3
4     public ReportDto toDto(PhishingReport r) {
5         // moi cho can "bay gio" deu hoi clock
6         return new ReportDto(..., ageLabel(
7             r.getReportedAt(), clock.instant()));
8     }
9 }
10
11 // Production: Clock.systemUTC()
12 // Trong test: dong ho dung yen tai mot thoi diem
13 Clock fixed = Clock.fixed(
14     Instant.parse("2026-07-14T10:00:00Z"),
15     ZoneOffset.UTC);
16 var service = new ReportingServiceImpl(fixed, ...);
```

Testability là tiêu chí thiết kế: code dễ test thường cũng là code ít phụ thuộc ẩn, dễ bảo trì — hai mặt của một đồng xu.

Thực hành 1 — Phủ kín StubAiClient.classify

Yêu cầu (làm tại lớp, theo cặp)

Mở `modules/ai/src/test/java/.../StubAiClientTest.java`, bổ sung để `classify` được phủ **cả 3 lớp** hành vi:

- 1 `classify_threat...`: payload chứa ≥ 2 tín hiệu ("otp", "password") — assert nhãn, confidence bằng AssertJ `isBetween`
- 2 `classify_spam...`: payload khuyến mãi có "%" hoặc "free" — nhãn "spam"
- 3 `classify_clean...`: nội dung vô hại — nhãn "clean", confidence đúng 0.92
- 4 `classify_nullPayload...`: payload = null — hành vi là gì? Đọc code trước, đoán, rồi viết test kiểm chứng
- 5 Chuyển các ca 1–3 thành **một** `@ParameterizedTest` + `@MethodSource`

Gợi ý: tín hiệu so khớp ở dạng **chữ thường** — thử "OTP" viết hoa xem test còn pass không?

Thực hành 2 — benchmark với danh sách rỗng

Khung có sẵn — Mockito sẽ học kỹ ở buổi 6, hôm nay dùng như công thức

```
1 @ExtendWith(MockitoExtension.class)
2 class AnalyticsServiceImplTest {
3     static final UUID TENANT_ID = UUID.fromString(
4         "11111111-1111-1111-1111-111111111111");
5     @Mock RiskScoreRepository riskScoreRepository;
6     // ... các @Mock khác như file test that ...
7     @InjectMocks AnalyticsServiceImpl service;
8
9     @BeforeEach void setUp() {
10         TenantContext.set(TENANT_ID.toString());
11     }
12     @AfterEach void tearDown() {
13         TenantContext.clear();
14     }
15
16     @Test
17     void benchmark_whenNoScores_returnsZero() {
18         when(riskScoreRepository
19             .findByTenantIdAndScope(
20                 TENANT_ID, RiskScope.DEPT))
21             .thenReturn(List.of());
22
23         assertThat(service.benchmark(RiskScope.DEPT))
24             .isZero();
25     }
26 }
```

Thực hành 3 — Chạy, đọc report, nộp minh chứng

- 1 Chạy test của hai module vừa sửa:

```
./gradlew :modules:ai:test \
:modules:analytics:test
```

- 2 Mở report: `modules/ai/build/reports/tests/test/index.html` — chụp màn hình trang tổng quan (số test, 0 failed)
- 3 Cố tình sửa một expected cho **sai** (vd "threat" → "spam"), chạy lại, đọc thông báo fail của AssertJ — rồi sửa về đúng
- 4 Mở report JaCoCo của module ai, ghi lại % line coverage của `StubAiClient` trước/sau khi thêm test

Vì sao bước 3 quan trọng?

Test chưa từng thấy fail là test chưa được kiểm chứng — phải biết nó *trông thế nào khi thất bại* mới tin được nó.

Bài nộp về nhà (gắn với Bài tập lớn — 30%)

Mỗi nhóm BTL nộp trước buổi 6

Viết **tối thiểu 10 unit test** cho module DigiShield mà nhóm đã đăng ký (hoặc ứng dụng nhóm tự chọn đã được GV duyệt).

Yêu cầu kỹ thuật:

- Đặt đúng vị trí `src/test/java`, cùng package, tên `*Test.java`; tên method theo mẫu `tenMethod_dieuKien_ketQua`
- Dùng AssertJ; có ít nhất 1 `assertThrows` (hoặc `assertThatThrownBy`) và 1 `@ParameterizedTest`
- Mỗi test theo AAA, kiểm **một** hành vi; test độc lập — chạy riêng lẻ vẫn pass
- Toàn bộ pass với `./gradlew test` (không cần Docker)

Nộp: link branch Git `feature/unit-test-<nhom>` + ảnh report HTML + bảng ánh xạ *test* ↔ *nhánh code* (như slide phân tích StubApiClient).

Tổng kết buổi 5

Đã học:

- JUnit 5 = Platform + Jupiter + Vintage
- Cài đặt: Maven, Gradle, cấu hình thật của DigiShield
- `@Test`, vòng đời, AAA, quy ước đặt tên
- Assertion JUnit + AssertJ fluent
- `@ParameterizedTest` với 3 nguồn dữ liệu
- Chạy Gradle/IDE, đọc report; testability

Buổi 6 — Mockito

`StubApiClient` test được “tay không” vì **không có dependency**. Nhưng `ReportingServiceImpl` cần `repository`, `publisher`... → cần **mock**: `@Mock`, `when/thenReturn`, `verify`, `ArgumentCaptor`.

Việc cần làm

Hoàn thành bài nộp 10 unit test **trước buổi 6**.

Tài liệu tham khảo

- [1] Slide bài giảng điện tử.
- [2] Paul Ammann & Jeff Offutt, *Introduction to Software Testing*, Cambridge University Press, 2008.
- [3] Glenford J. Myers, *The Art of Software Testing*, 2nd edition, John Wiley & Sons, 2004.
- [4] ISTQB Foundation Level Syllabus, version 4.0, 2023. <https://istqb.org/>
- [5] JUnit Team, *JUnit 5 User Guide*,
<https://junit.org/junit5/docs/current/user-guide/>
- [6] AssertJ Core documentation, <https://assertj.github.io/doc/>
- [7] Gradle, *Testing in Java & JVM projects* (JVM Test Suites),
https://docs.gradle.org/current/userguide/java_testing.html
- [8] Mã nguồn DigiShield: `modules/ai`, `modules/analytics`, `modules/reporting`,
`boot/app/build.gradle.kts`.